Práctica Semana 10: Árboles AVL

Rotaciones, balanceo e implementación

Apellidos y nombres: _____

Código: _____

Escuela profesional: _____

Universidad Nacional del Altiplano – Puno, noviembre de 2026

Contenido

1 Actividad 1: AVL completo en C++17 **2**

1.1 Preguntas de análisis 6

2 Actividad 2 **6**

2.1 2A: Eliminación AVL 6

2.2 2B: BST vs. AVL en Python 6

3 Actividad 3: SIGA-UNAP-AVL **11**

4 Repositorio **11**

1 Actividad 1: AVL completo en C++17

Se implementaron `rotar_derecha`, `rotar_izquierda`, `balancear` (casos LL, RR, LR, RL) e `insertar_avl`. El factor de balance es $fb = h(der) - h(izq)$.

Listing 1: avl.h

```
#pragma once
#include <memory>
#include <algorithm>
#include <string>
#include <utility>

template<typename K, typename V = K>
struct NodoBST {
    K clave; V valor;
    std::unique_ptr<NodoBST> izq, der;
    int altura;
    explicit NodoBST(K k, V v=V{})
        : clave(std::move(k)), valor(std::move(v)),
          izq(nullptr), der(nullptr), altura(1) {}
};

template<typename K,typename V>
int altura(const std::unique_ptr<NodoBST<K,V>>& n)
{ return n ? n->altura : 0; }

template<typename K,typename V>
int factor_balance(const std::unique_ptr<NodoBST<K,V>>& n)
{ return n ? altura(n->der)-altura(n->izq) : 0; }

template<typename K,typename V>
void actualizar_altura(std::unique_ptr<NodoBST<K,V>>& n)
{ if(n) n->altura=1+std::max(altura(n->izq),altura(n->der)); }

/* TODO 1: rotar_derecha(z)
          z                 y
         / \               / \
        y   C     -->     A   z
       / \                   / \
      A   B                 B   C
*/
template<typename K,typename V>
void rotar_derecha(std::unique_ptr<NodoBST<K,V>>& z) {
    std::unique_ptr<NodoBST<K,V>> y = std::move(z->izq);
    z->izq = std::move(y->der);      // B pasa a ser hijo izquierdo de z
    actualizar_altura(z);            // z queda abajo: actualizar primero
    y->der = std::move(z);           // z pasa a ser hijo derecho de y
    z = std::move(y);                // y es la nueva raíz del subárbol
    actualizar_altura(z);
}

/* TODO 2: rotar_izquierda(z)  (simétrica)
        z                     y
       / \                   / \
      A   y       -->       z   C
         / \               / \
        B   C             A   B
*/
template<typename K,typename V>
void rotar_izquierda(std::unique_ptr<NodoBST<K,V>>& z) {
    std::unique_ptr<NodoBST<K,V>> y = std::move(z->der);
    z->der = std::move(y->izq);
    actualizar_altura(z);
    y->izq = std::move(z);
```

```
    z = std::move(y);
    actualizar_altura(z);
}

// TODO 3: balancear(n) -- casos LL, RR, LR, RL
// fb = altura(der) - altura(izq)
template<typename K,typename V>
void balancear(std::unique_ptr<NodoBST<K,V>>& n) {
    actualizar_altura(n);
    int fb = factor_balance(n);
    if (fb < -1) {                              // cargado a la izquierda
        if (factor_balance(n->izq) > 0)         // caso LR
            rotar_izquierda(n->izq);
        rotar_derecha(n);                       // caso LL (o LR tras 1a rotacion)
    } else if (fb > 1) {                        // cargado a la derecha
        if (factor_balance(n->der) < 0)         // caso RL
            rotar_derecha(n->der);
        rotar_izquierda(n);                     // caso RR (o RL tras 1a rotacion)
    }
}

// TODO 4: insertar_avl
template<typename K,typename V>
void insertar_avl(std::unique_ptr<NodoBST<K,V>>& raiz,
                  K clave, V valor=V{}) {
    if (!raiz) {
        raiz = std::make_unique<NodoBST<K,V>>(std::move(clave), std::move(valor));
        return;
    }
    if (clave < raiz->clave)
        insertar_avl(raiz->izq, std::move(clave), std::move(valor));
    else if (raiz->clave < clave)
        insertar_avl(raiz->der, std::move(clave), std::move(valor));
    else {
        raiz->valor = std::move(valor);         // clave repetida: actualiza valor
        return;
    }
    balancear(raiz);
}

// YA IMPLEMENTADA --- no modificar
template<typename K,typename V>
NodoBST<K,V>* buscar(const std::unique_ptr<NodoBST<K,V>>& raiz,
                     const K& clave) {
    if (!raiz) return nullptr;
    if (clave==raiz->clave) return raiz.get();
    if (clave<raiz->clave) return buscar(raiz->izq,clave);
    return buscar(raiz->der,clave);
}

// ---------------- Actividad 2A: eliminacion ----------------
template<typename K,typename V>
NodoBST<K,V>* minimo(const std::unique_ptr<NodoBST<K,V>>& n) {
    return n->izq ? minimo(n->izq) : n.get();
}

template<typename K,typename V>
void eliminar_avl(std::unique_ptr<NodoBST<K,V>>& raiz,
                  const K& clave) {
    if (!raiz) return;
    if (clave < raiz->clave)
        eliminar_avl(raiz->izq, clave);
    else if (clave > raiz->clave)
        eliminar_avl(raiz->der, clave);
    else {
```

```
        if (!raiz->izq && !raiz->der) {
            raiz = nullptr;                                    // Caso 1: hoja
        } else if (!raiz->izq) {
            raiz = std::move(raiz->der);                       // Caso 2: solo hijo
                der
        } else if (!raiz->der) {
            raiz = std::move(raiz->izq);                       // Caso 2: solo hijo
                izq
        } else {                                               // Caso 3: dos hijos
            NodoBST<K,V>* suc = minimo(raiz->der);             // sucesor inorden
            K clave_suc = suc->clave;                          // copia (el nodo se
                destruira)
            raiz->clave = suc->clave;
            raiz->valor = suc->valor;
            eliminar_avl(raiz->der, clave_suc);
        }
    }
    if (raiz) balancear(raiz);                                 // reequilibrar
}
```

Listing 2: avl_main.cpp (pruebas P1–P6 y prueba extra)

```
#include <iostream>
#include <cassert>
#include <cstdlib>
#include <random>
#include "avl.h"

// Función auxiliar: retorna altura del árbol
template<typename K,typename V>
int h(const std::unique_ptr<NodoBST<K,V>>& r) { return r?r->altura:0; }

// Verifica el invariante AVL (|fb|<=1, alturas consistentes y orden BST)
template<typename K,typename V>
bool es_avl(const std::unique_ptr<NodoBST<K,V>>& n) {
    if (!n) return true;
    int hi = altura(n->izq), hd = altura(n->der);
    if (n->altura != 1 + std::max(hi, hd)) return false;
    if (std::abs(hd - hi) > 1) return false;
    if (n->izq && !(n->izq->clave < n->clave)) return false;
    if (n->der && !(n->clave < n->der->clave)) return false;
    return es_avl(n->izq) && es_avl(n->der);
}

void test_P1_insercion_simple() {
    // Insertar 3 claves ordenadas --- en BST sería h=3; en AVL h=2
    std::unique_ptr<NodoBST<int,int>> r=nullptr;
    insertar_avl(r,10,10);
    insertar_avl(r,20,20);
    insertar_avl(r,30,30); // provoca rotación RR (rotar_izquierda)
    assert(h(r)==2 && "P1: altura debe ser 2 tras RR rotation");
    assert(r->clave==20 && "P1: raiz debe ser 20");
    std::cout << "[P1 OK] insercion_simple" << std::endl;
}

void test_P2_caso_LR() {
    std::unique_ptr<NodoBST<int,int>> r=nullptr;
    insertar_avl(r,30,30);
    insertar_avl(r,10,10);
    insertar_avl(r,20,20); // provoca rotacion LR
    assert(r->clave==20 && "P2: raiz debe ser 20");
    assert(h(r)==2 && "P2: altura debe ser 2");
    std::cout << "[P2 OK] caso_LR" << std::endl;
}

```

```
44 void test_P3_insercion_ordenada_grande() {
45     // Insertar 1..15 en orden --- BST degeneraría h=15
46     std::unique_ptr<NodoBST<int,int>> r=nullptr;
47     for(int i=1;i<=15;i++) insertar_avl(r,i,i);
48     assert(h(r)<=4 && "P3: altura debe ser <= 4 para 15 nodos AVL");
49     assert(buscar(r,8) && "P3: nodo 8 debe existir");
50     std::cout << "[P3 OK] insercion_ordenada_grande h=" << h(r) << std::endl;
51 }
52
53 void test_P4_eliminar_hoja() {
54     std::unique_ptr<NodoBST<int,int>> r=nullptr;
55     for(int x:{20,10,30}) insertar_avl(r,x,x);
56     eliminar_avl(r,10); // hoja izquierda
57     assert(!buscar(r,10) && "P4: nodo 10 debe haberse eliminado");
58     assert(h(r)<=2 && "P4: árbol sigue balanceado");
59     std::cout << "[P4 OK] eliminar_hoja" << std::endl;
60 }
61
62 void test_P5_eliminar_raiz_dos_hijos() {
63     std::unique_ptr<NodoBST<int,int>> r=nullptr;
64     for(int x:{20,10,30,25,35}) insertar_avl(r,x,x);
65     eliminar_avl(r,20); // nodo raiz con 2 hijos
66     assert(!buscar(r,20) && "P5: nodo 20 eliminado");
67     assert(h(r)<=3 && "P5: árbol sigue balanceado");
68     std::cout << "[P5 OK] eliminar_raiz_dos_hijos" << std::endl;
69 }
70
71 void test_P6_eliminar_con_rebalanceo() {
72     // Insertar 1..7, eliminar 1 y verificar balanceo
73     std::unique_ptr<NodoBST<int,int>> r=nullptr;
74     for(int i=1;i<=7;i++) insertar_avl(r,i,i);
75     int h_antes = h(r);
76     eliminar_avl(r,1);
77     // Verificar que el árbol sigue siendo AVL
78     assert(h(r)<=h_antes && "P6: altura no aumentó tras eliminación");
79     std::cout << "[P6 OK] eliminar_con_rebalanceo" << std::endl;
80 }
81
82 // Prueba extra (no pedida): 2000 inserciones y 1000 eliminaciones aleatorias
83 void test_extra_invariante_aleatorio() {
84     std::mt19937 gen(42);
85     std::unique_ptr<NodoBST<int,int>> r=nullptr;
86     for (int i=0;i<2000;i++) {
87         insertar_avl(r, (int)(gen()%5000), i);
88         assert(es_avl(r) && "extra: invariante AVL tras insertar");
89     }
90     for (int i=0;i<1000;i++) {
91         eliminar_avl(r, (int)(gen()%5000));
92         assert(es_avl(r) && "extra: invariante AVL tras eliminar");
93     }
94     std::cout << "[EXTRA OK] invariante AVL en insercion/eliminacion aleatoria" <<
        std::endl;
95 }
96
97 int main() {
98     test_P1_insercion_simple();
99     test_P2_caso_LR();
100    test_P3_insercion_ordenada_grande();
101    std::cout << "=== Todos los casos P1-P3 pasaron ===" << std::endl;
102    test_P4_eliminar_hoja();
103    test_P5_eliminar_raiz_dos_hijos();
104    test_P6_eliminar_con_rebalanceo();
105    test_extra_invariante_aleatorio();
106    std::cout << "=== Todos los casos P4-P6 pasaron ===" << std::endl;
107    return 0;
```

```
}
```

Salida de la ejecución

Compilado con `g++ -std=c++17 -Wall -Wextra`:

```
[P1 OK] insercion_simple
[P2 OK] caso_LR
[P3 OK] insercion_ordenada_grande h=4
=== Todos los casos P1-P3 pasaron ===
[P4 OK] eliminar_hoja
[P5 OK] eliminar_raiz_dos_hijos
[P6 OK] eliminar_con_rebalanceo
[EXTRA OK] invariante AVL en insercion/eliminacion aleatoria
=== Todos los casos P4-P6 pasaron ===
```

1.1 Preguntas de análisis

P1-A. Al insertar 10, 20, 30 ocurre **1 rotación**, de tipo **RR** (simple a la izquierda sobre el 10). La raíz queda en 20 y la altura es 2. (El comentario del enunciado dice «LL», pero la secuencia crece hacia la derecha, por lo que el caso correcto es RR.)

P1-B. Al insertar 30, 10, 20 ocurren **2 rotaciones**, de tipo **LR** (doble): primero `rotar_izquierda` sobre 10 y luego `rotar_derecha` sobre 30. La raíz queda en 20.

P1-C. El mínimo de nodos de un AVL de altura h cumple $N(h) = N(h-1) + N(h-2) + 1$, con $N = 1, 2, 4, 7, 12, 20, \ldots$ Como $12 \leq 15 < 20$, un AVL de 15 nodos tiene altura entre 4 y **5**. P3 acepta $h \leq 4$ porque insertar 1..15 en orden produce un árbol perfecto ($2^4 - 1 = 15$), que es la altura mínima posible; es un caso favorable, no la cota general (que sería 5).

2 Actividad 2

2.1 2A: Eliminación AVL

`eliminar_avl` (incluida en `avl.h`, listado anterior) maneja los tres casos del BST (hoja, un hijo, dos hijos con sucesor inorden) y llama a `balancear` al volver de la recursión. Pasa P4–P6 y una prueba extra con 2000 inserciones y 1000 eliminaciones aleatorias verificando el invariante AVL tras cada operación.

2.2 2B: BST vs. AVL en Python

Listing 3: avl_python.py

```python
import random
import math
import sys
from dataclasses import dataclass
from time import perf_counter


class Nodo:
    def __init__(self, clave, valor=None):
        self.clave = clave
        self.valor = valor
        self.izq = self.der = None
        self.altura = 1

```

```python

# ---------------- Funciones AVL ----------------
def height(n):
    return n.altura if n else 0


def fb(n):
    """Factor de balance = altura(der) - altura(izq)."""
    return height(n.der) - height(n.izq) if n else 0


def actualizar(n):
    n.altura = 1 + max(height(n.izq), height(n.der))


def rotar_der(z):
    y = z.izq
    z.izq = y.der
    y.der = z
    actualizar(z)
    actualizar(y)
    return y


def rotar_izq(z):
    y = z.der
    z.der = y.izq
    y.izq = z
    actualizar(z)
    actualizar(y)
    return y


def balancear(n):
    actualizar(n)
    f = fb(n)
    if f < -1:                          # cargado a la izquierda
        if fb(n.izq) > 0:               # caso LR
            n.izq = rotar_izq(n.izq)
        return rotar_der(n)             # caso LL
    if f > 1:                           # cargado a la derecha
        if fb(n.der) < 0:               # caso RL
            n.der = rotar_der(n.der)
        return rotar_izq(n)             # caso RR
    return n


def insertar_avl(raiz, clave, valor=None):
    if raiz is None:
        return Nodo(clave, valor)
    if clave < raiz.clave:
        raiz.izq = insertar_avl(raiz.izq, clave, valor)
    elif clave > raiz.clave:
        raiz.der = insertar_avl(raiz.der, clave, valor)
    else:
        raiz.valor = valor              # clave repetida: actualiza el valor
        return raiz
    return balancear(raiz)


def buscar(raiz, clave):
    while raiz is not None:
        if clave == raiz.clave:
            return raiz
        raiz = raiz.izq if clave < raiz.clave else raiz.der
```

```
    return None


# ---------------- BST simple ----------------
def insertar_bst(raiz, clave):  # BST simple sin balanceo
    if raiz is None:
        return Nodo(clave)
    if clave < raiz.clave:
        raiz.izq = insertar_bst(raiz.izq, clave)
    elif clave > raiz.clave:
        raiz.der = insertar_bst(raiz.der, clave)
    return raiz


def altura_arbol(n):
    if n is None:
        return 0
    return 1 + max(altura_arbol(n.izq), altura_arbol(n.der))


def experimento(n, orden='random'):
    claves = list(range(1, n + 1))
    if orden == 'random':
        random.shuffle(claves)
    # BST simple
    raiz_bst = None
    sys.setrecursionlimit(n + 100)
    for c in claves:
        raiz_bst = insertar_bst(raiz_bst, c)
    h_bst = altura_arbol(raiz_bst)
    # AVL
    raiz_avl = None
    for c in claves:
        raiz_avl = insertar_avl(raiz_avl, c)
    h_avl = raiz_avl.altura if raiz_avl else 0
    limite_teorico = int(1.44 * math.log2(n + 1))
    return h_bst, h_avl, limite_teorico


def tabla_2b():
    random.seed(2026)  # reproducible
    print(f"{'n':>8} | {'BST (ord)':>10} | {'AVL (ord)':>10} | {'Teorico':>10} | {'
        BST (rnd)':>10} | {'AVL (rnd)':>10}")
    print('-' * 68)
    for n in [100, 500, 1000, 5000, 10000]:
        h_bst_o, h_avl_o, teo = experimento(n, 'ordered')
        h_bst_r, h_avl_r, _ = experimento(n, 'random')
        print(f"{n:>8} | {h_bst_o:>10} | {h_avl_o:>10} | {teo:>10} | {h_bst_r:>10}
            | {h_avl_r:>10}")


# ---------------- Actividad 3: SIGA-UNAP-AVL ----------------
@dataclass
class EstudianteUNAP:
    codigo: str
    nombre: str
    escuela: str
    promedio: float


class SIGA_AVL:
    def __init__(self):
        self.raiz = None
        self.total = 0

```

```
    def matricular(self, est: EstudianteUNAP):
        t0 = perf_counter()
        existe = buscar(self.raiz, est.codigo) is not None
        self.raiz = insertar_avl(self.raiz, est.codigo, est)
        if not existe:
            self.total += 1
        t1 = perf_counter()
        accion = 'Actualizado' if existe else 'Matriculado'
        print(f'  {accion} {est.nombre} en {(t1 - t0) * 1e6:.2f} µs')

    def buscar_estudiante(self, codigo: str):
        t0 = perf_counter()
        nodo = buscar(self.raiz, codigo)
        t1 = perf_counter()
        if nodo:
            print(f'  Encontrado: {nodo.valor} ({(t1 - t0) * 1e6:.2f} µs)')
        else:
            print(f'  Código {codigo} no encontrado ({(t1 - t0) * 1e6:.2f} µs)')

    def listar_ordenado(self):
        # recorrido inorden (iterativo) => lista ordenada por código
        pila, actual, i = [], self.raiz, 0
        if actual is None:
            print('  (sin estudiantes)')
            return
        print(f"  {'#':>3} {'Código':<12}{'Nombre':<16}{'Escuela':<10}{'Prom.':>6}"
            )
        while pila or actual:
            while actual:
                pila.append(actual)
                actual = actual.izq
            actual = pila.pop()
            e = actual.valor
            i += 1
            print(f'  {i:>3} {e.codigo:<12}{e.nombre:<16}{e.escuela:<10}{e.promedio
                :>6.1f}')
            actual = actual.der

    def estadisticas(self):
        h = self.raiz.altura if self.raiz else 0
        teo = int(1.44 * math.log2(self.total + 1)) if self.total > 0 else 0
        print(f'  Total estudiantes: {self.total}')
        print(f'  Altura AVL actual: {h}')
        print(f'  Altura teórica max: {teo}')
        print(f'  Eficiencia: {"ÓPTIMO" if h <= teo else "SUBÓPTIMO"}')


MUESTRA = [
    EstudianteUNAP('2026100001', 'Ana Quispe', 'Sistemas', 15.5),
    EstudianteUNAP('2026100002', 'Luis Mamani', 'Sistemas', 14.0),
    EstudianteUNAP('2026100003', 'Rosa Condori', 'IA&CD', 17.2),
    EstudianteUNAP('2026100004', 'Ivan Huanca', 'Sistemas', 13.8),
    EstudianteUNAP('2026100005', 'Maria Apaza', 'IA&CD', 16.9),
    EstudianteUNAP('2026100006', 'Carlos Tito', 'Sistemas', 12.1),
    EstudianteUNAP('2026100007', 'Lucia Ramos', 'IA&CD', 18.0),
    EstudianteUNAP('2026100008', 'Pedro Vargas', 'Sistemas', 15.0),
    EstudianteUNAP('2026100009', 'Sofia Luna', 'IA&CD', 16.4),
    EstudianteUNAP('2026100010', 'Juan Puma', 'Sistemas', 14.7),
]


def menu():
    siga = SIGA_AVL()
    print('Cargando 10 estudiantes...')
    for e in MUESTRA:
```

```
        siga.matricular(e)

    while True:
        print('\n=== SIGA-UNAP-AVL ===')
        print('1. Matricular nuevo estudiante')
        print('2. Buscar por código')
        print('3. Listar todos (inorden)')
        print('4. Estadísticas del árbol')
        print('5. Salir')
        try:
            op = input('Opción: ').strip()
        except EOFError:
            break
        if op == '1':
            cod = input('  Código: ').strip()
            nom = input('  Nombre: ').strip()
            esc = input('  Escuela: ').strip()
            try:
                prom = float(input('  Promedio: '))
            except ValueError:
                print('  Promedio inválido, operación cancelada.')
                continue
            siga.matricular(EstudianteUNAP(cod, nom, esc, prom))
        elif op == '2':
            siga.buscar_estudiante(input('  Código: ').strip())
        elif op == '3':
            siga.listar_ordenado()
        elif op == '4':
            siga.estadisticas()
        elif op == '5':
            break
        else:
            print('  Opción no válida.')


if __name__ == '__main__':
    if len(sys.argv) > 1 and sys.argv[1] == '--tabla':
        tabla_2b()          # solo la tabla de la Actividad 2B
    else:
        tabla_2b()
        menu()
```

Resultados (semilla aleatoria 2026):

n	BST (ord)	AVL (ord)	Teórico	BST (rnd)	AVL (rnd)
100	100	7	9	12	8
500	500	9	12	18	11
1000	1000	10	14	20	12
5000	5000	13	17	27	15
10000	10000	14	19	29	16

P2B-A. La altura del BST con inserción ordenada es exactamente n: el árbol se degenera en una lista enlazada y la búsqueda cuesta $O(n)$.

P2B-B. No. $1.44 \log_2(10001) \approx 19.13$, es decir, límite 19. El AVL mide 14 (ordenado) y 16 (aleatorio): 5 y 3 niveles por debajo.

P2B-C. La premisa no es exacta: las alturas no son idénticas (14 vs. 16 con $n = 10000$), pero ambas son $O(\log n)$ y muy cercanas. El AVL mantiene $|fb| \leq 1$ en cada nodo mediante rotaciones,

sin importar el orden de llegada. La inserción ordenada da un árbol casi perfecto ($\lceil \log_2(n+1) \rceil$); la aleatoria queda algo más alta porque el AVL tolera desbalances de 1.

3 Actividad 3: SIGA-UNAP-AVL

La clase `SIGA_AVL` (en `avl_python.py`) usa el AVL con el código de estudiante como clave. Ofrece matricular, buscar (mostrando μs), listar en inorden y estadísticas. Un código repetido actualiza los datos en lugar de duplicar. Ejemplo de ejecución (carga de 10 estudiantes, luego opciones 3, 2, 1 y 4):

```
   # Código      Nombre          Escuela    Prom.
   1 2026100001  Ana Quispe      Sistemas    15.5
   2 2026100002  Luis Mamani     Sistemas    14.0
   ...
  10 2026100010  Juan Puma       Sistemas    14.7
  Total estudiantes: 10
  Altura AVL actual: 4
  Altura teórica max: 4
  Eficiencia: ÓPTIMO

  Encontrado: EstudianteUNAP(codigo='2026100007', nombre='Lucia Ramos', ...) (2.23
     μs)
  Código 999 no encontrado (1.50 μs)
  Matriculado Leo Cruz  -> Total 11, altura 4, teórica 5, ÓPTIMO
```

Los tiempos en µs varían en cada ejecución.

4 Repositorio

Código fuente y README en GitHub: `https://github.com/<usuario>/avl_apellido`